

АННОТАЦИЯ

Выпускная квалификационная работа посвящена разработке интернет-платформы «ГородМинут», предназначенной для размещения, поиска и бронирования лофтов с применением механизма динамического ценообразования. В работе проведён анализ научных подходов к динамическому ценообразованию, рассмотрены практики его использования в электронной коммерции и сервисных цифровых платформах, исследованы бизнес-процессы взаимодействия арендатора и арендодателя в условиях переменной цены. На основе результатов анализа разработана модель веб-приложения, реализующая расчёт стоимости аренды в зависимости от календарных и временных факторов. Выполнено проектирование архитектуры системы, структуры базы данных и пользовательского интерфейса, а также программная реализация серверной и клиентской частей платформы с использованием Java 17, Spring Boot, PostgreSQL, React, TypeScript и сопутствующих технологий. Практическая значимость работы заключается в создании готовой цифровой платформы, позволяющей автоматизировать сдачу площадок в аренду и повысить управляемость ценообразования.

Общий объем работы составляет 40 страниц основного текста, 2 страницы приложений, 8 рисунков, 10 использованных источников.

Ключевые слова: динамическое ценообразование, цифровая платформа, электронная торговая площадка, веб-приложение, аренда лофтов, Java, Spring Boot, React, PostgreSQL, JWT, Docker.

СОДЕРЖАНИЕ

АННОТАЦИЯ.....	2
СОДЕРЖАНИЕ	3
СПИСОК ИСПОЛЬЗУЕМЫХ СОКРАЩЕНИЙ	4
ВВЕДЕНИЕ.....	5
1 АНАЛИЗ Подходов к динамическому ценообразованию и практик его использования в цифровом бизнесе	8
1.1 Сущность динамического ценообразования и его место в цифровой коммерции	8
1.2 Анализ подходов к динамическому ценообразованию в коммерческих предприятиях	10
1.3 Практики использования динамического ценообразования в бизнесе.....	12
1.4 Анализ бизнес-процессов с использованием динамического ценообразования в цифровых платформах	14
2 ПРОЕКТИРОВАНИЕ интернет-платформы с динамическим ценообразованием	17
2.1 Формирование функциональных и нефункциональных требований.....	17
2.2 Разработка модели интернет-платформы с динамическим ценообразованием	19
2.3 Проектирование архитектуры системы	22
2.4 Проектирование базы данных.....	25
2.5 Проектирование пользовательского интерфейса.....	26
3 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ Веб-платформы с динамическим ценообразованием.	32
3.1 Выбор и обоснование использования инструментальных средств разработки.....	32
3.2 Программная реализация серверной части платформы.....	34
3.3 Реализация клиентской части платформы	36
ЗАКЛЮЧЕНИЕ	40
СПИСОК ЛИТЕРАТУРЫ.....	41
ПРИЛОЖЕНИЯ	44

СПИСОК ИСПОЛЬЗУЕМЫХ СОКРАЩЕНИЙ

API — программный интерфейс приложения

DTO — объект передачи данных

JWT — JSON Web Token

REST — архитектурный стиль взаимодействия распределённых систем

UI — пользовательский интерфейс

UX — пользовательский опыт взаимодействия с системой

СУБД — система управления базами данных

ЭТП — электронная торговая площадка

БД — база данных

ПО — программное обеспечение

ВВЕДЕНИЕ

В условиях цифровизации экономики ценовой механизм перестаёт быть статичным элементом коммерческой политики и превращается в инструмент оперативного управления спросом, загрузкой ресурсов и доходностью. Для цифровых платформ, обеспечивающих взаимодействие поставщика услуги и конечного потребителя в режиме реального времени, способность корректировать цену в зависимости от контекста сделки становится одним из важнейших факторов конкурентоспособности. Особенно отчётливо данная тенденция проявляется в сфере аренды пространств, где стоимость услуги зависит от дня недели, времени суток, продолжительности бронирования, сезонности и текущей загрузки объекта.

Традиционный подход к формированию цены в сервисном бизнесе основан на фиксированных тарифах, которые пересматриваются дискретно и, как правило, вручную. Такой подход, хотя и отличается простотой, не учитывает фактическую вариативность спроса и приводит либо к недополучению выручки в периоды повышенной востребованности, либо к снижению конверсии в периоды низкого спроса. В отличие от него динамическое ценообразование предполагает адаптивное изменение цены на основании заранее определённых правил, аналитических показателей или прогнозных моделей. Следовательно, внедрение механизмов динамического ценообразования в цифровые платформы позволяет повысить экономическую эффективность эксплуатации ресурса при сохранении прозрачности расчёта для пользователя.

Для российского рынка аренды лофтов и площадок под мероприятия данная задача имеет прикладное значение. С одной стороны, значительная часть объектов предлагается через разрозненные каналы коммуникации, где расчёт стоимости осуществляется вручную, что увеличивает операционные издержки и замедляет обработку заявок. С другой стороны, пользователи ожидают мгновенного получения информации о доступности площадки, итоговой стоимости аренды и статусе заявки. Тем самым возникает необходимость в

разработке интернет-платформы, объединяющей каталог объектов, личные кабинеты участников сделки, механизмы обработки заявок и модуль расчёта цены, способный автоматически учитывать параметры конкретного бронирования.

Разрабатываемая в рамках настоящей работы интернет-платформа получила название «ГородМинут». По своему функциональному назначению она представляет собой веб-приложение, обеспечивающее размещение лофтов арендодателями, поиск и просмотр площадок арендаторами, подачу заявок на аренду и расчёт стоимости с использованием набора временных надбавок. Реализованный механизм динамического ценообразования не опирается на персонализированное профилирование пользователей или модели машинного обучения; напротив, он построен на детерминированных правилах, что делает его объяснимым, проверяемым и практически применимым для малого и среднего бизнеса.

Целью выпускной квалификационной работы является реализация электронной торговой площадки с динамическим ценообразованием. Для достижения поставленной цели в работе решаются следующие задачи: провести анализ подходов к динамическому ценообразованию в коммерческих предприятиях; провести анализ практик использования динамического ценообразования в бизнесе; провести анализ бизнес-процессов с использованием динамического ценообразования в цифровых платформах; разработать модель интернет-платформы с динамическим ценообразованием; реализовать цифровую платформу с динамическим ценообразованием на основе полученной модели.

Объектом исследования является процесс цифрового взаимодействия участников рынка краткосрочной аренды площадок в рамках электронной платформы. Предметом исследования выступают методы организации динамического ценообразования и программной реализации веб-приложения,

обеспечивающего расчёт стоимости аренды, управление заявками и поддержку ролевого взаимодействия пользователей.

Научная новизна работы состоит в адаптации принципов динамического ценообразования к предметной области аренды лофтов посредством разработки прикладной модели веб-платформы, сочетающей прозрачный расчёт стоимости, календарную блокировку подтверждённых интервалов и разделение пользовательских ролей. Практическая значимость обусловлена тем, что созданное решение может быть использовано как основа для коммерческого сервиса аренды площадок, а также как прототип для дальнейшего развития модулей прогнозирования спроса, персонализации и расширенной аналитики.

1 АНАЛИЗ ПОДХОДОВ К ДИНАМИЧЕСКОМУ ЦЕНООБРАЗОВАНИЮ И ПРАКТИК ЕГО ИСПОЛЬЗОВАНИЯ В ЦИФРОВОМ БИЗНЕСЕ

1.1 Сущность динамического ценообразования и его место в цифровой коммерции

Под динамическим ценообразованием в современной литературе обычно понимается механизм изменения цены товара или услуги во времени под воздействием параметров спроса, предложения, поведения потребителей, состояния конкурентов и характеристик самого ресурса. В отличие от статического тарифообразования, при котором цена устанавливается на длительный период и изменяется эпизодически, динамическая модель ориентирована на непрерывную или периодическую адаптацию ценового предложения. Данный подход тесно связан с разными концепциями максимизации прибыли, поскольку во всех перечисленных случаях цена рассматривается как управляемая переменная, позволяющая влиять на загрузку ресурса и финансовый результат.

Первоначально методы динамического ценообразования получили широкое распространение в отраслях с ограниченной и невозполнимой ёмкостью: авиаперевозках, гостиничном бизнесе, аренде автомобилей и билетных сервисах. Экономическая логика здесь заключается в том, что неиспользованный ресурс в определённый временной интервал теряет коммерческую ценность. Так, непроданное место в самолёте после вылета или незабронированный гостиничный номер в прошедшую дату уже не могут быть реализованы. По мере развития платформенной экономики аналогичные механизмы были перенесены в электронную коммерцию, маркетплейсы услуг, доставку, такси, краткосрочную аренду недвижимости и пространства для мероприятий.

В цифровой среде динамическое ценообразование опирается на данные, собираемые системой в процессе её функционирования. Источниками таких данных выступают календарь бронирований, история покупок, длительность сессии, поведение пользователя на сайте, текущая загрузка объекта, внешние события, сезонные пики, ценовая политика конкурентов и особенности конкретного временного интервала. Чем выше уровень цифровой зрелости платформы, тем более тонко она способна дифференцировать цену. Однако рост сложности модели неизбежно повышает требования к прозрачности логики расчёта, вычислительной инфраструктуре и соблюдению этических ограничений.

С точки зрения прикладной реализации можно выделить три укрупнённых класса механизмов динамического ценообразования. Первый класс составляют модели, основанные на заранее заданных правилах и коэффициентах. Они относительно просты, хорошо объяснимы и подходят для задач, где необходимо гарантировать предсказуемость и управляемость результата. Ко второму классу относятся аналитические модели, использующие статистические зависимости между ценой и ключевыми параметрами спроса. Третий класс образуют интеллектуальные модели, применяющие методы машинного обучения, обучения с подкреплением и прогнозирования временных рядов. Выбор конкретного подхода определяется масштабом бизнеса, доступностью данных, требованиями к интерпретируемости и допустимым уровнем автоматизации.

В контексте исследуемой предметной области аренды лофтов подход на основе набора заданных заранее правил представляется методологически оправданным. С одной стороны, он позволяет быстро внедрить цифровой механизм изменения цены без накопления больших объёмов исторических данных. С другой стороны, арендодатель получает возможность непосредственно контролировать правила формирования стоимости, задавая надбавки за периоды пикового спроса. Наконец, для пользователя такая модель остаётся понятной: изменение цены объясняется объективными параметрами

времени бронирования, а не скрытыми факторами, что положительно влияет на доверие к платформе.

1.2 Анализ подходов к динамическому ценообразованию в коммерческих предприятиях

В коммерческой практике динамическое ценообразование реализуется через совокупность организационных и вычислительных процедур, направленных на установление экономически обоснованной цены в конкретный момент времени. Наиболее распространённым является подход, основанный на сегментации факторов ценообразования. В рамках данного подхода выделяются внутренние факторы, зависящие от самого предприятия, и внешние факторы, формируемые рынком. К внутренним факторам относятся себестоимость услуги, целевая маржинальность, пропускная способность ресурса, регламенты обслуживания и стратегические ограничения. К внешним — колебания спроса, цены конкурентов, сезонность, поведение потребителей и макроэкономическая конъюнктура.

Первый практически значимый подход можно охарактеризовать как календарно-временной. Он предполагает, что цена определяется на основании принадлежности бронируемого интервала к определённой временной категории: будний день, выходной день, предпраздничный период, вечернее время, ночной интервал и т.п. Достоинством данного подхода является высокая прозрачность и технологическая простота. Кроме того, он хорошо соотносится с реальными паттернами потребления услуги, когда спрос закономерно возрастает по пятницам, выходным дням или в удобные для мероприятий вечерние часы.

Второй подход связан с управлением загрузкой. Цена возрастает по мере приближения фактической загрузки ресурса к предельной ёмкости или, напротив, снижается в целях стимулирования спроса при низкой занятости. Данный механизм типичен для авиации, гостиниц и агрегаторов поездок. Для его

корректной работы необходимо постоянное измерение коэффициента загрузки и наличие формализованной функции отклика цены на изменение загрузки. В небольших сервисных бизнесах этот подход часто комбинируется с календарным методом, что позволяет избежать чрезмерной волатильности цен.

Третий подход предполагает учёт конкурентной среды. Предприятие в автоматическом или полуавтоматическом режиме сопоставляет собственные цены с рыночными аналогами и корректирует стоимость для сохранения позиции в ценовом диапазоне. В электронной коммерции данная практика получила распространение в интернет-ритейле, где цены тысяч товарных позиций сравниваются с прайсами конкурентов. Однако для рынков услуг, особенно локальных и нишевых, применение этого подхода ограничено различиями в характеристиках объектов и трудоёмкостью сопоставления предложений.

Четвёртый подход носит персонализированный характер и использует индивидуальные признаки пользователя: историю покупок, предпочтения, географическое положение, вероятность совершения сделки и ценовую чувствительность. Несмотря на потенциальную эффективность, он является наиболее дискуссионным, поскольку может восприниматься клиентами как несправедливый, особенно если пользователи обнаруживают различие цен без очевидного объяснения. Для выпускной квалификационной работы и разрабатываемой платформы использование персонализированного подхода нецелесообразно, так как поставленная задача ориентирована на прозрачный и юридически нейтральный механизм ценообразования.

С методологической точки зрения можно утверждать, что для малого и среднего бизнеса наиболее рациональной стратегией внедрения динамического ценообразования является постепенный переход от простых правил к более сложной аналитике. Иначе говоря, на первом этапе достаточно задать набор формализованных коэффициентов, отражающих факторы наиболее высокого влияния. Затем, по мере накопления статистики, возможно введение

дополнительных параметров: длительности бронирования, среднего чека, конверсии по временным слотам, отказов по цене и повторных обращений. Такой эволюционный сценарий позволяет не перегружать систему на старте и в то же время сохраняет потенциал для её дальнейшего развития.

Для платформы «ГородМинут» применён именно такой поэтапный подход. Базовая цена задаётся арендодателем для конкретного объекта, а динамический компонент рассчитывается автоматически на основе фиксированных надбавок. На текущем этапе выделены следующие условия: пятница — надбавка 10 %, суббота и воскресенье — 20 %, вечернее время с 18:00 до 23:00 — 10 %, ночное время с 23:00 до 06:00 — 20 %. Данный набор правил отражает специфику рынка аренды лофтов, где наибольший спрос приходится на вечерние и выходные интервалы.

1.3 Практики использования динамического ценообразования в бизнесе

Современный бизнес демонстрирует широкий спектр сценариев применения динамического ценообразования. Одним из наиболее известных примеров является рынок пассажирских перевозок, где цена билета определяется комбинацией времени покупки, ожидаемой загрузки рейса, класса обслуживания и сезонного периода. Близкие по логике механизмы применяются в гостиничной индустрии, где стоимость номера может изменяться ежедневно и даже несколько раз в сутки. В обоих случаях основной задачей является не только максимизация дохода, но и выравнивание спроса по времени.

На рынке платформенных сервисов особенно заметна практика динамического ценообразования в агрегаторах такси и доставки. Повышающие коэффициенты активируются при дисбалансе между спросом и предложением, неблагоприятных погодных условиях, дорожной перегруженности или дефиците исполнителей в отдельном районе. Несмотря на иногда критическую реакцию пользователей, бизнес-логика такого механизма очевидна: цена сигнализирует

рынку о повышенной ценности ресурса в конкретный момент и стимулирует поставщиков услуги выходить на платформу.

В электронной коммерции динамическое ценообразование используется как на маркетплейсах, так и в интернет-магазинах. Там оно может учитывать акционные периоды, уровень складских остатков, ценовые действия конкурентов, поведение пользователя и показатели конверсии карточки товара. В отличие от сервисных рынков, где объект продажи связан со временем оказания услуги, товарные платформы чаще опираются на конкурентов и историю спроса. Тем не менее общая идея остаётся неизменной: цена перестаёт быть фиксированным атрибутом и становится результатом вычислимой управленческой политики.

Отдельного внимания заслуживает рынок краткосрочной аренды недвижимости и пространств. Здесь динамическое ценообразование применяется для выравнивания доходности в зависимости от дня недели, туристического сезона, городских мероприятий, срока проживания или времени бронирования. Для аренды лофтов и площадок под мероприятия особенно важны временные паттерны спроса, так как пользователи чаще всего бронируют объект на вечернее время, выходные или праздничные даты. Следовательно, внедрение даже сравнительно простой модели надбавок позволяет приблизить стоимость аренды к реальной коммерческой ценности временного слота.

Однако практика показывает, что эффективность динамического ценообразования определяется не только формулой расчёта, но и качеством пользовательской коммуникации. Если цена изменяется непрозрачно, пользователь склонен воспринимать платформу как манипулятивную. В этой связи важнейшим принципом становится объяснимость. Пользователь должен понимать, какие параметры влияют на стоимость, почему один временной слот дороже другого и в какой момент формируется итоговая цена. Данный принцип особенно важен для сервисов начального уровня развития, которые ещё только формируют доверие своей аудитории.

Таким образом, анализ бизнес-практик позволяет сделать два вывода. Во-первых, динамическое ценообразование является не узкоспециализированным, а универсальным инструментом цифровой коммерции. Во-вторых, для сервисных платформ на ранних стадиях жизненного цикла наиболее оправдано использование прозрачных моделей на основе заранее прописанных правил. Именно такой вариант реализован в исследуемой платформе, поскольку он обеспечивает баланс между экономической гибкостью, технологической реализуемостью и пользовательской понятностью.

1.4 Анализ бизнес-процессов с использованием динамического ценообразования в цифровых платформах

Цифровая платформа, использующая динамическое ценообразование, представляет собой не просто программную систему расчёта стоимости, а среду координации нескольких взаимосвязанных бизнес-процессов. В рассматриваемой предметной области ключевыми субъектами являются гость платформы, арендатор, арендодатель и сама платформа как цифровой посредник. Взаимодействие между ними строится вокруг жизненного цикла заявки: обнаружение объекта, анализ характеристик площадки, выбор временного интервала, расчёт цены, отправка запроса, принятие решения владельцем и фиксация результата в календаре доступности.

Первый процесс можно обозначить как информационно-поисковый. Пользователь без регистрации просматривает каталог лофтов, карточки объектов, фотографии, описание, контакты и доступные временные слоты. На этом этапе динамическое ценообразование выполняет не столько доходную, сколько коммуникативную функцию: система позволяет ещё до отправки заявки показать пользователю предварительную стоимость с учётом выбранной даты и времени. Тем самым снижается число нерелевантных обращений и повышается качество пользовательского решения.

Второй процесс связан с оформлением заявки. После выбора даты и времени пользователь вводит имя и телефон, а затем передаёт заявку в систему. Именно здесь происходит окончательный расчёт стоимости по формализованным правилам. Следовательно, модуль динамического ценообразования должен быть встроен в транзакционный контур платформы, а не существовать отдельно как вспомогательный калькулятор. Иначе говоря, рассчитанная стоимость становится частью делового объекта «заявка» и используется в дальнейшем арендодателем при принятии решения.

Третий процесс относится к управлению предложением со стороны арендодателя. Для владельца площадки платформа выступает инструментом ведения цифрового витрины и входящего потока заявок. Арендодатель регистрируется через сценарий добавления площадки, после чего получает возможность создавать новые объекты, просматривать поступившие заявки и изменять их статус. В момент подтверждения заявки соответствующий интервал времени блокируется в календаре, что предотвращает двойное бронирование. Следовательно, динамическое ценообразование здесь тесно связано с механизмом резервирования ресурса.

Четвёртый процесс имеет административно-аналитический характер. Хотя в текущей версии платформы он реализован в ограниченном объёме, уже на данном этапе система накапливает данные, потенциально пригодные для дальнейшей аналитики: востребованность отдельных дней недели, популярность временных слотов, частота отклонения заявок, соотношение новых и повторных обращений. В перспективе именно эти данные могут стать основанием для перехода от детерминированной модели надбавок к более сложному механизму оптимизации цены.

Исследование бизнес-процессов позволяет сформулировать ключевые требования к платформе. Во-первых, расчёт цены должен выполняться автоматически и воспроизводимо при каждом выборе интервала. Во-вторых, итоговая стоимость должна сохраняться в заявке как юридически и операционно

значимая информация. В-третьих, система должна обеспечивать согласованность между ценообразованием и календарной доступностью объекта. В-четвёртых, интерфейс должен быть спроектирован так, чтобы логика ценообразования воспринималась пользователем как понятная и справедливая.

Выводы по первой главе. Проведённый анализ показал, что динамическое ценообразование является эффективным инструментом управления доходностью в цифровых сервисах, особенно там, где услуга привязана ко времени использования ресурса. Для предметной области аренды лофтов на текущем этапе наиболее обоснованным является применение модели, основанной на календарных и временных надбавках. Практики бизнеса подтверждают, что даже простая и прозрачная модель изменения цены позволяет лучше согласовать стоимость услуги с фактическим спросом.

2 ПРОЕКТИРОВАНИЕ ИНТЕРНЕТ-ПЛАТФОРМЫ С ДИНАМИЧЕСКИМ ЦЕНООБРАЗОВАНИЕМ

2.1 Формирование функциональных и нефункциональных требований

Проектирование интернет-платформы начинается с формализации требований, отражающих как потребности пользователей, так и технологические ограничения разработки. Поскольку разрабатываемая система является веб-приложением электронной торговой площадки, требования целесообразно разделить на функциональные и нефункциональные. Функциональные требования описывают действия, которые система должна обеспечивать, тогда как нефункциональные характеризуют качество, надёжность и безопасность решения.

Для незарегистрированного пользователя, то есть гостя платформы, необходимо обеспечить просмотр каталога лофтов, переход в карточку конкретной площадки, просмотр фотографий, описания и контактной информации, а также выбор предполагаемой даты и времени аренды. Существенным требованием является немедленный расчёт стоимости с учётом надбавок, что позволяет ещё до регистрации или авторизации оценить коммерческую привлекательность объекта. Кроме того, гость должен иметь возможность отправить заявку на аренду, указав минимально необходимый набор данных: имя и телефон.

Для арендатора после регистрации требуется реализация базового сценария учётной записи. В текущей версии платформы арендатор регистрируется по адресу электронной почты и паролю, после чего получает доступ к защищённым пользовательским функциям. Несмотря на то что набор возможностей арендатора пока ограничен, сам факт наличия авторизации является архитектурно важным, поскольку создаёт основу для последующего

расширения сценариев: истории заявок, избранных площадок, уведомлений и повторных бронирований.

Для арендодателя система должна предоставлять более широкий набор функций. Регистрация осуществляется через сценарий «Добавить площадку», в результате чего создаётся как пользовательский аккаунт владельца, так и первый объект размещения. После авторизации арендодатель получает доступ к добавлению новых площадок, просмотру входящих заявок, индикации количества новых обращений и действиям по принятию либо отклонению заявки. При подтверждении заявки соответствующий временной интервал должен быть помечен как недоступный, что обеспечивает целостность бизнес-процесса бронирования.

К числу нефункциональных требований относятся надёжность вычисления цены, безопасность хранения учётных данных, разграничение прав доступа, корректная валидация входной информации, модульность архитектуры и возможность контейнеризованного развёртывания. Так как система обрабатывает контактные данные пользователей и управляет доступом к личным кабинетам, обязательными являются аутентификация, авторизация и защита API от несанкционированных запросов. Для прикладной реализации это означает необходимость использования фреймворка безопасности, токенов доступа и валидации пользовательских данных на серверной стороне.

Отдельным требованием выступает объяснимость модуля динамического ценообразования. Алгоритм расчёта должен быть однозначным, тестируемым и таким образом встроенным в архитектуру, чтобы изменение правил не приводило к нарушению остальных компонентов системы. Именно поэтому ценообразование в проекте оформляется как часть бизнес-логики серверного приложения, а не как вспомогательный фронтенд-скрипт. Такой подход обеспечивает единообразный результат расчёта вне зависимости от клиента, обращающегося к API.

2.2 Разработка модели интернет-платформы с динамическим ценообразованием

Модель платформы «ГородМинут» можно представить как совокупность трёх взаимосвязанных уровней: предметно-логического, прикладного и пользовательского. На предметно-логическом уровне определяются основные сущности предметной области — площадка, пользователь, заявка, временной интервал и правило надбавки. На прикладном уровне фиксируются сервисы и сценарии обработки данных. На пользовательском уровне формируются интерфейсы взаимодействия с системой для различных ролей.

Центральной сущностью платформы является площадка, или лофт. Для неё задаются идентификатор, название, описание, базовая стоимость аренды, адрес, контактные данные, набор изображений и привязка к владельцу. Базовая цена выступает отправной точкой для дальнейшего динамического расчёта. С ней связана сущность заявки, которая хранит данные арендатора, выбранную дату, временной диапазон, рассчитанную стоимость и текущий статус обработки. Тем самым заявка становится не просто сообщением о намерении арендовать объект, а структурированной записью делового процесса.

Сущность пользователя в модели платформы является ролевой. Минимально выделяются две прикладные роли: арендатор и арендодатель. На уровне реализации они определяют доступ к соответствующим API-методам и интерфейсным сценариям. При этом арендодатель связан с одной или несколькими площадками, а арендатор — с собственными учётными данными и потенциальной историей действий. Для управления безопасностью необходимы также атрибуты, связанные с аутентификацией: адрес электронной почты, хеш пароля и роль.

Особое место в модели занимает механизм динамического ценообразования. Формально его можно описать как функцию, принимающую базовую цену и набор временных признаков выбранного интервала и возвращающую итоговую стоимость. В простейшем виде такая функция

реализуется посредством последовательного применения процентных надбавок. Если обозначить базовую стоимость через P_0 , а множество применимых надбавок через k_i , где k_i выражено в долях единицы, итоговая цена может быть записана как $P = P_0 \times \prod(1 + k_i)$. В рамках данной работы для упрощения пользовательского восприятия используется суммарная модель надбавок, при которой применимые проценты аккумулируются, а итоговая цена определяется как $P = P_0 \times (1 + \sum k_i)$.

Выбор суммарной модели обусловлен двумя обстоятельствами. Во-первых, пользователю легче интерпретировать итоговую цену, если каждая надбавка добавляется к базовой стоимости как самостоятельный фактор. Во-вторых, для предметной области аренды лофтов число одновременно действующих правил ограничено и не требует более сложных способов композиции коэффициентов. Например, если бронирование приходится на субботу в 20:00, то к базовой цене одновременно применяются надбавка за выходной день и надбавка за вечерний интервал. В результате цена формируется детерминированно и прозрачно.

С точки зрения процессной модели модуль ценообразования вызывается в двух ключевых точках: при предварительном расчёте стоимости в пользовательском интерфейсе и при создании заявки на сервере. Дублирование точки вызова имеет принципиальное значение. Интерфейсный расчёт повышает удобство пользования, однако юридически и технически значимым должен считаться серверный результат, поскольку только он формируется в доверенной среде и сохраняется в базе данных. Таким образом обеспечивается консистентность данных и устойчивость к манипуляциям со стороны клиента.

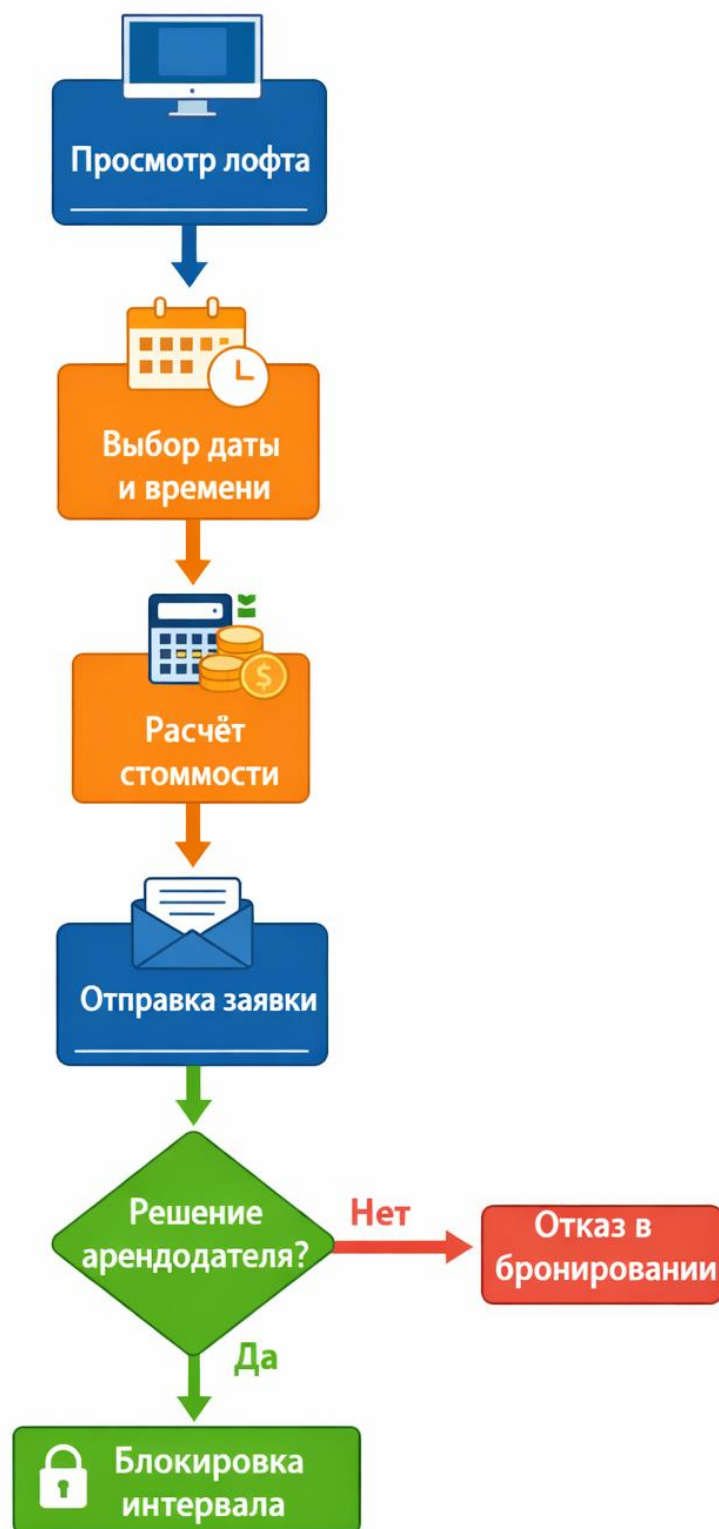


Рисунок 2.1 – Блок-схема процесса бронирования

Модель платформы предусматривает также связь между статусом заявки и календарной доступностью площадки. Пока заявка находится в состоянии ожидания, временной интервал может рассматриваться как условно доступный или временно резервируемый в зависимости от выбранной бизнес-политики. После подтверждения арендодателем интервал блокируется, и новые заявки на этот период не должны приниматься. При отклонении заявки интервал возвращается в пул доступных. Такая логика является обязательной частью общей модели, поскольку цена и доступность ресурса в сервисной платформе неразрывно связаны.

2.3 Проектирование архитектуры системы

Архитектурно платформа реализована как клиент-серверное веб-приложение. Серверная часть отвечает за хранение и обработку данных, выполнение бизнес-правил, аутентификацию пользователей и предоставление REST API. Клиентская часть обеспечивает отображение пользовательского интерфейса, навигацию между страницами, ввод данных и взаимодействие с API. В качестве хранилища используется реляционная база данных PostgreSQL, а контейнеризация сервисов осуществляется посредством Docker и Docker Compose.

На серверной стороне ключевую роль играет модульная организация исходного кода. Пакет `controller` содержит REST-эндпоинты, принимающие HTTP-запросы и возвращающие структурированные ответы. Пакет `service` реализует бизнес-логику платформы, включая расчёт стоимости, обработку заявок и операции над площадками. Пакет `model` описывает сущности предметной области и отображение на таблицы базы данных. Пакет `repository` инкапсулирует доступ к данным через интерфейсы Spring Data JPA. Пакет `config` предназначен для конфигурирования безопасности, JWT-аутентификации и других инфраструктурных аспектов. Наконец, пакет `dto` содержит объекты

запросов и ответов, позволяющие отделить внутренние сущности от внешнего контракта API.

Подобное разделение имеет методическое значение. Во-первых, оно снижает связанность компонентов и повышает сопровождаемость системы. Во-вторых, оно делает логику приложения более прозрачной для тестирования и расширения. Например, изменение правил динамического ценообразования локализуется преимущественно в сервисном слое, не затрагивая контроллеры и интерфейс хранения данных. Следовательно, архитектура соответствует принципам разделения ответственности и слоистого проектирования.

Клиентская часть организована по аналогичному принципу модульности. В директории `pages` размещаются верхнеуровневые страницы приложения: каталог, карточка площадки, форма регистрации, кабинет арендодателя и экран заявок. В директории `components` находятся переиспользуемые элементы интерфейса: карточки лофтов, формы, кнопки, бейджи, карточки заявок и блоки расчёта стоимости. Директория `api` содержит функции взаимодействия с REST API, инкапсулирующие HTTP-запросы. В директории `types` располагаются типы TypeScript, обеспечивающие строгую типизацию данных, поступающих с сервера и используемых в компонентах.

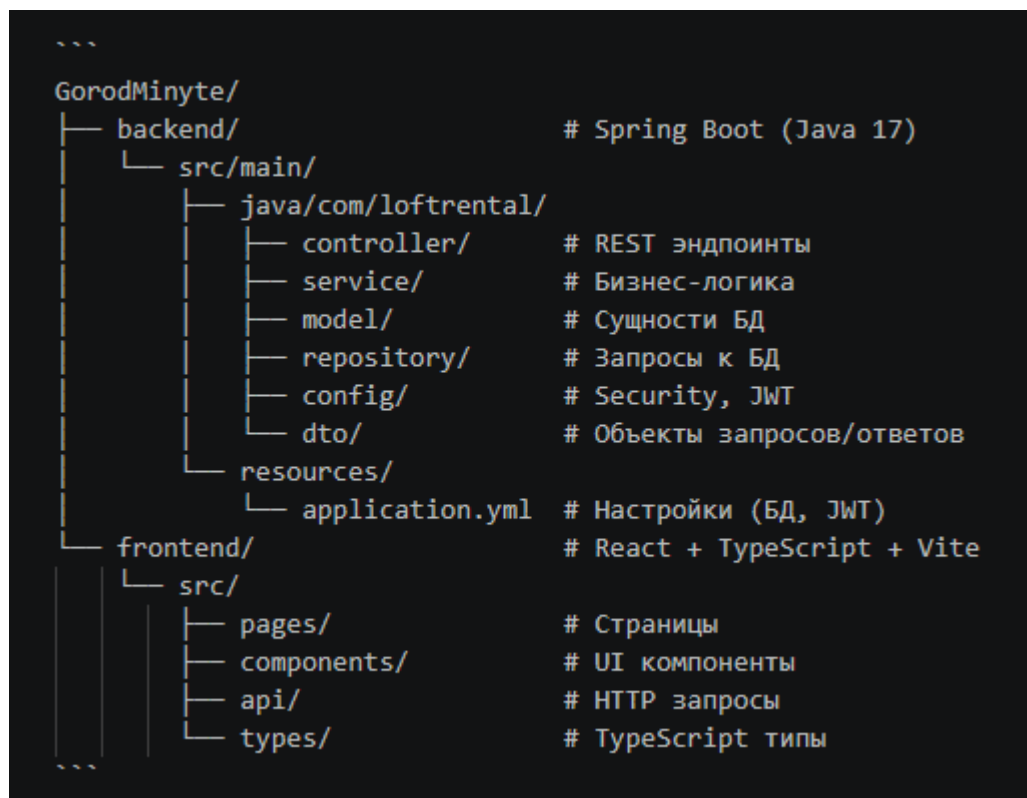


Рисунок 2.2 – архитектурная схема проекта

На уровне взаимодействия компонентов можно выделить следующий сценарий. Пользователь открывает страницу карточки площадки и выбирает дату и время. React-компонент фиксирует изменение состояния формы и либо локально иницирует предварительный расчёт, либо обращается к серверному API-калькулятору. После отправки заявки клиент вызывает соответствующий эндпоинт backend-приложения. Контроллер передаёт данные сервису, сервис валидирует их, вычисляет стоимость, формирует запись заявки и сохраняет её в PostgreSQL. Затем результат возвращается клиенту в виде DTO, а интерфейс отображает подтверждение создания заявки.

Системная архитектура дополняется инфраструктурным контуром. Docker Compose позволяет совместно запускать backend, frontend, базу данных PostgreSQL и, при необходимости, Nginx как прокси-сервер для раздачи статических файлов фронтенда в производственной среде. Контейнерный подход обеспечивает воспроизводимость окружения и упрощает перенос приложения между разработческим и эксплуатационным стендами.

2.4 Проектирование базы данных

Реляционная модель данных платформы строится вокруг сущностей User, Loft и BookingRequest. Таблица User хранит сведения о зарегистрированных участниках системы: идентификатор, email, хеш пароля, имя и роль. Роль определяет тип доступа пользователя и используется в механизме авторизации. Для арендодателя запись пользователя связана с одной или несколькими площадками, размещёнными в системе. Для арендатора данная сущность обеспечивает возможность последующего расширения личного кабинета и истории заявок.

Таблица Loft содержит информацию о сдаваемых площадках. В её состав входят название, описание, адрес, контактный телефон, базовая цена, ссылка на владельца, а также поля, относящиеся к медиа- и презентационным характеристикам объекта. С точки зрения предметной области именно Loft является единицей коммерческого предложения. Базовая цена хранится как исходное значение, от которого рассчитывается стоимость конкретного интервала аренды. При необходимости структура таблицы может быть расширена полями вместимости, типа площадки, доступного оборудования и географических координат.

Таблица BookingRequest предназначена для фиксации заявок на аренду. Она включает ссылку на площадку, имя заявителя, телефон, дату, время начала и окончания аренды, рассчитанную стоимость, статус заявки и отметку времени создания записи. Важно подчеркнуть, что итоговая стоимость сохраняется непосредственно в заявке, а не вычисляется заново при каждом обращении к записи. Такое решение обеспечивает неизменность коммерчески значимых данных и позволяет в будущем анализировать историю ценообразования.

Дополнительная логика БД связана с обеспечением уникальности и согласованности временных интервалов. Хотя в простейшей реализации блокировка может выполняться на уровне бизнес-логики приложения, корректная модель предусматривает проверку пересечения подтверждённых

бронирований по площадке и дате. При масштабировании системы целесообразно расширить схему отдельной сущностью `ReservedSlot` или `BookingInterval`, где будут храниться именно заблокированные интервалы с привязкой к заявке и текущему статусу. Это упростит работу календаря доступности и сделает систему более устойчивой к конкурентным обращениям.

Важной особенностью проектирования базы данных является выбор корректных типов полей. Для денежной величины следует использовать тип с фиксированной точностью, исключающий ошибки округления, характерные для чисел с плавающей точкой. Для временных атрибутов рекомендуется раздельное хранение даты и времени либо использование `datetime`-типов в зависимости от логики расчёта. Применение ограничений `not null`, внешних ключей и индексов по часто используемым полям повышает надёжность хранения и производительность доступа.

2.5 Проектирование пользовательского интерфейса

Пользовательский интерфейс платформы должен одновременно решать две задачи: обеспечивать быстрое выполнение целевых сценариев и демонстрировать прозрачность логики формирования стоимости. В связи с этим при проектировании интерфейса особое внимание уделяется карточке площадки, поскольку именно в ней пересекаются визуальная презентация объекта, выбор временного интервала и расчёт цены. Интерфейс должен минимизировать когнитивную нагрузку пользователя, предоставляя информацию последовательно: сначала характеристики лофта, затем параметры бронирования, далее итоговую стоимость и форму заявки.

Главная страница и каталог выполняют функцию первичного отбора вариантов. Здесь важны понятные карточки объектов, содержащие фотографию, название, краткое описание и базовую цену.

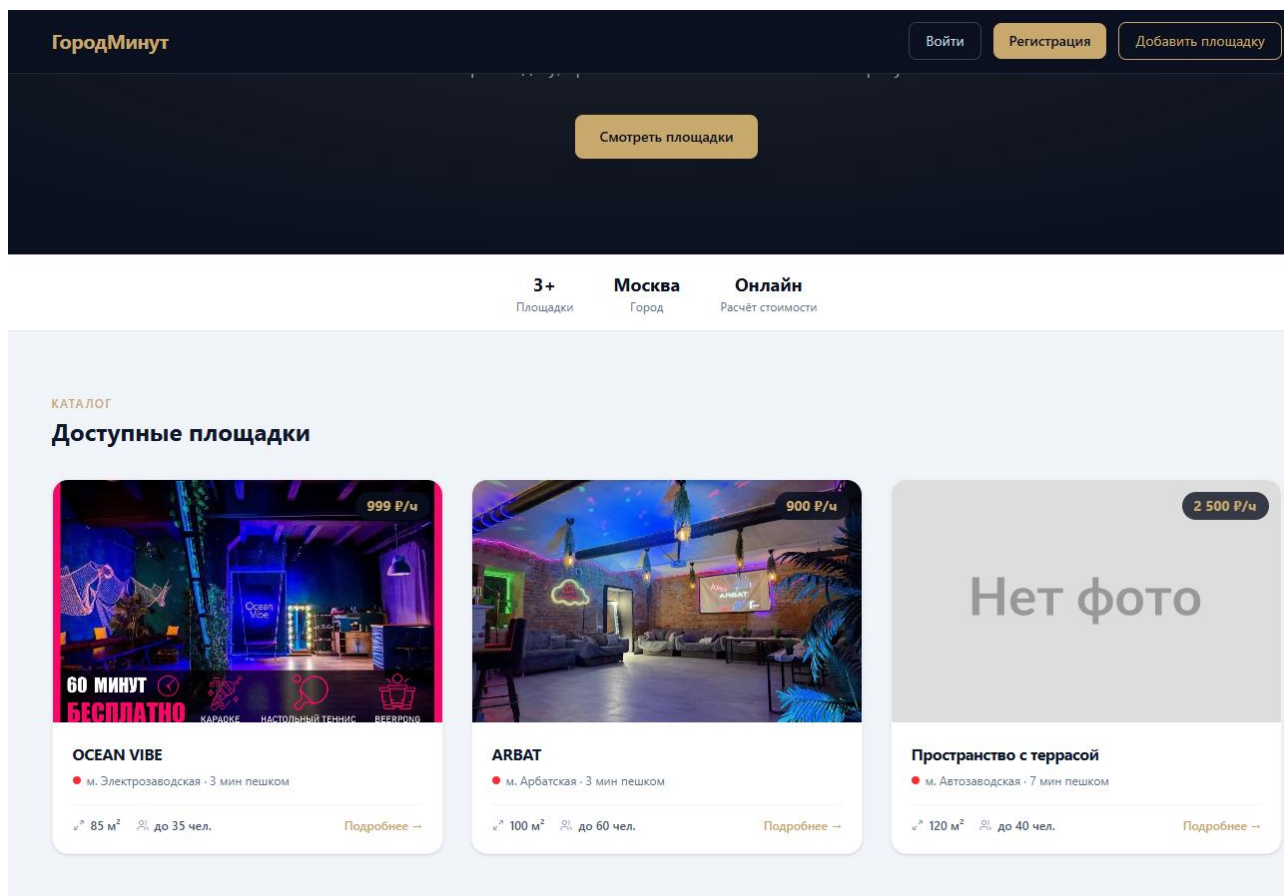


Рисунок 2.3 – Макет главной страницы

При переходе в карточку площадки пользователь получает расширенную информацию, включая контакты и форму выбора даты и времени. После ввода параметров система должна немедленно пересчитать стоимость и показать, какие надбавки были применены. Такой способ представления не только повышает удобство, но и реализует принцип объяснимости динамического ценообразования.



OCEAN VIBE

Площадь
85 м²

Вместимость
до 35 чел.

Цена/час
999 Р

ОПИСАНИЕ

Двухуровневый лофт в морском стиле. Предназначен для дней рождения, вечеринок, семинаров, детских праздников и свадеб. В аренду входят: звуковое оборудование, микшерный пульт, настольный теннис, настольный футбол, кондиционер, кухонные принадлежности. Высота потолков 4,5 м. Размеры 8×10 м. Собственная парковка.

КОНТАКТЫ

ул. Малая Семеновская, 5/1

м. Электрозаводская • 3 мин пешком

+7 (495) 790-46-77

info@loftmsk.ru

ВЫБРАТЬ ДАТУ И ВРЕМЯ

Рисунок 2.4 – Карточка площадки

Добавить площадку ×

Email
email@example.com

Пароль (мин. 6 символов)

Название площадки
LOFT SPACE

Город
Москва

Телефон
+7 (999) 000-00-00

Станция метро
Курская

Минут от метро
5

Адрес
ул. Пушкина, д. 1

Мест (макс.)
30

Площадь (м²)
80

Цена за час (₽)
1500

Email площадки (необязательно)
info@loft.ru

Рисунок 2.5 – Форма для арендодателя

Для арендодателя приоритетным является интерфейс управления площадками и заявками. Кнопка добавления новой площадки должна быть

логически обособлена и визуально заметна, так как она отражает ключевую бизнес-операцию владельца. Раздел входящих заявок должен содержать список обращений с основными атрибутами: объект, дата, время, стоимость, контактные данные и статус. Дополнительный бейдж количества новых заявок повышает оперативность реакции.

The interface is divided into several sections:

- Calendar:** Shows dates from 20 to 30. The date 26 is highlighted with a blue bar.
- Time Selection:** A grid of time slots from 00:00 to 23:00. The start time is set to 23:00. The selected time range is 21:00 — 23:00.
- Pricing Summary:** A dark blue box containing the base price (1998.0 rub), surcharges for weekends (+20%) and evening time (+10%), the date (19 апреля 2026 г.), and the final price (2 597,4 Р).
- Contact Form:** Fields for Name, Phone, Email, and Telegram, followed by a submit button.

Рисунок 2.6 – Окно бронирования и расчёта стоимости

С точки зрения UX важна и роль ограничений. Пользователь не должен иметь возможность отправить заявку на уже заблокированное время, указать некорректный диапазон часов или оставить обязательные поля пустыми. Часть таких ограничений реализуется на уровне клиентской валидации, однако определяющая проверка должна выполняться на сервере. Таким образом достигается баланс между удобством пользовательского ввода и надёжностью бизнес-логики.

3 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ ВЕБ-ПЛАТФОРМЫ С ДИНАМИЧЕСКИМ ЦЕНООБРАЗОВАНИЕМ.

3.1 Выбор и обоснование использования инструментальных средств разработки

Выбор технологического стека для реализации платформы определялся требованиями к модульности, скорости разработки, безопасности, удобству сопровождения и распространённости инструментов в современной веб-разработке. Серверная часть построена на языке Java 17 и фреймворке Spring Boot 3.2.3. Данное сочетание обеспечивает зрелую экосистему библиотек, строгость типизации, высокую устойчивость к расширению проекта и наличие проверенных решений для REST API, работы с данными и безопасности.

Использование Spring Web обусловлено необходимостью построения серверного API, обслуживающего запросы клиентского приложения. Компонент Spring Data JPA выбран для абстрагирования доступа к базе данных и сокращения объёма шаблонного кода при выполнении типовых операций чтения и записи. Применение Spring Security оправдано потребностью в разграничении доступа между ролями пользователя, защите закрытых маршрутов и интеграции аутентификации на основе токенов. Библиотека Spring Validation обеспечивает декларативную проверку входных данных и уменьшает вероятность попадания некорректных значений в бизнес-логику.

В качестве СУБД используется PostgreSQL, поскольку данная система сочетает надёжность, развитые механизмы транзакционной обработки, поддержку сложных запросов и широкую распространённость в веб-проектах. Для хранения и передачи токенов аутентификации применяется JWT-библиотека jjwt версии 0.12.3. Она позволяет реализовать stateless-аутентификацию, при которой серверу не требуется хранить сессионное состояние пользователя.

Библиотека Lombok выбрана в качестве средства сокращения шаблонного кода для моделей и DTO, а Maven используется как стандартная система сборки и управления зависимостями Java-проекта.

Клиентская часть реализована на React 19 с использованием TypeScript. Выбор React продиктован компонентной моделью, удобством управления состоянием и широкими возможностями построения динамических пользовательских интерфейсов. TypeScript усиливает надёжность проекта за счёт статической типизации и уменьшает вероятность ошибок при обмене данными между API и компонентами интерфейса. В качестве инструмента сборки выбран Vite 8, обеспечивающий быстрый запуск среды разработки и эффективную сборку фронтенд-приложения.

Для стилизации используется TailwindCSS 4, позволяющий строить согласованную систему визуальных компонентов с минимизацией ручного CSS-кода. Навигация между страницами реализована средствами React Router 7. Для выполнения HTTP-запросов к API используется Axios, обеспечивающий удобную настройку базовых URL, заголовков и перехватчиков запросов. Библиотека Lucide React отвечает за отображение иконок, а утилиты clsx, tailwind-merge и class-variance-authority упрощают управление классами и вариативностью визуальных компонентов. Дополнительно используется ESLint, предназначенный для поддержания единообразия и качества фронтенд-кода.

Инфраструктурный слой платформы базируется на Docker и Docker Compose. Контейнеризация позволяет зафиксировать зависимости и конфигурацию сервисов, исключив различия между окружениями разработки и демонстрации. Для базы данных используется образ PostgreSQL 16. В производственной конфигурации фронтенд может обслуживаться через Nginx, который выступает как сервер статических файлов и точка маршрутизации внешнего трафика. В совокупности выбранный стек формирует технологически целостное решение, подходящее для прикладной ВКР и демонстрирующее современный подход к разработке веб-платформ.

3.2 Программная реализация серверной части платформы

Серверная часть приложения организована как многослойная Spring Boot-система. Точка входа и инфраструктурная конфигурация определяют жизненный цикл приложения, регистрацию бинов и подключение внешних компонентов. На уровне контроллеров реализованы REST-эндпоинты, отвечающие за регистрацию пользователей, аутентификацию, получение каталога площадок, получение данных конкретного лофта, создание заявок и управление ими со стороны арендодателя. Каждый контроллер оперирует DTO-объектами, что позволяет скрыть внутреннюю структуру сущностей и обеспечивать стабильность внешнего контракта API.

Сервисный слой содержит основную бизнес-логику. Здесь реализуются операции создания и редактирования площадок, расчёт итоговой стоимости аренды, проверка корректности временного интервала, формирование записи заявки и смена её статуса. Именно в сервисном слое сосредоточен алгоритм динамического ценообразования. Он получает базовую стоимость площадки, анализирует выбранную дату и время и последовательно определяет применимые надбавки. Такая организация кода обеспечивает централизованный контроль над коммерческой логикой и делает её независимой от способа вызова — через веб-интерфейс, тесты или возможное мобильное приложение в будущем.

Алгоритм расчёта стоимости можно описать следующим образом. На вход подаются дата бронирования, время начала, время окончания и базовая цена площадки. Сервис определяет день недели, затем проверяет, относится ли временной интервал к вечернему или ночному периоду. После этого вычисляется совокупная надбавка. Если дата приходится на пятницу, применяется коэффициент 0,10; если на субботу или воскресенье — 0,20. Далее, если выбранный интервал пересекается с диапазоном 18:00–23:00, добавляется 0,10; если с диапазоном 23:00–06:00 — 0,20. Итоговая цена сохраняется в заявке и возвращается клиенту.

```

12 public class PricingService {
13
14     private static final LocalTime EVENING_START = LocalTime.of(hour: 18, minute: 0); 1 usage
15     private static final LocalTime EVENING_END = LocalTime.of(hour: 23, minute: 0); 1 usage
16     private static final LocalTime NIGHT_START = LocalTime.of(hour: 23, minute: 0); 1 usage
17     private static final LocalTime NIGHT_END = LocalTime.of(hour: 6, minute: 0); 1 usage
18
19     public record PriceResult(double basePrice, double totalPrice, String breakdown) {} 3 usages
20
21     @ public PriceResult calculate(LocalDate date, LocalTime startTime, LocalTime endTime, double baseRate) {
22         List<String> parts = new ArrayList<>();
23         double multiplier = 1.0;
24
25         DayOfWeek dow = date.getDayOfWeek();
26
27         if (dow == DayOfWeek.FRIDAY) {
28             multiplier += 0.10;
29             parts.add("пятница +10%");
30         }
31         if (dow == DayOfWeek.SATURDAY || dow == DayOfWeek.SUNDAY) {
32             multiplier += 0.20;
33             parts.add("выходной +20%");
34         }
35
36         if (hasEveningOverlap(startTime, endTime)) {
37             multiplier += 0.10;
38             parts.add("вечернее время 18:00-23:00 +10%");
39         }
40         if (hasNightOverlap(startTime, endTime)) {
41             multiplier += 0.20;
42             parts.add("ночное время 23:00-06:00 +20%");
43         }
44
45         double hours = calculateHours(startTime, endTime);
46         double basePrice = Math.round(baseRate * hours * 100.0) / 100.0;
47         double totalPrice = Math.round(basePrice * multiplier * 100.0) / 100.0;
48
49         String breakdown = parts.isEmpty()
50             ? "Базовая цена: " + basePrice + " руб."
51             : "Базовая цена: " + basePrice + " руб. | " + String.join(delimiter: ", ", parts);
52
53         return new PriceResult(basePrice, totalPrice, breakdown);
54     }
55
56     @ private double calculateHours(LocalTime start, LocalTime end) { 1 usage
57         int startMin = start.getHour() * 60 + start.getMinute();
58         int endMin = end.getHour() * 60 + end.getMinute();
59         if (endMin <= startMin) endMin += 24 * 60;
60         return (endMin - startMin) / 60.0;
61     }
62
63     private boolean hasEveningOverlap(LocalTime start, LocalTime end) { 1 usage
64         return overlaps(start, end, EVENING_START, EVENING_END);
65     }

```

Рисунок 3.1 – Фрагмент кода расчёта надбавок

С точки зрения инженерной корректности важно, что сервер не доверяет клиентскому расчёту. Даже если фронтенд предварительно показывает пользователю стоимость, окончательное значение вычисляется повторно на

backend-стороне. Тем самым исключается возможность подмены цены путём изменения клиентского JavaScript-кода или отправки искусственно модифицированного запроса. Данный принцип особенно важен для систем, в которых цена является коммерчески значимым параметром.

Модуль безопасности построен на Spring Security и JWT. При регистрации и авторизации пользователя система формирует токен доступа, содержащий сведения о роли и идентификаторе субъекта. В последующих запросах токен передаётся в заголовке Authorization и проверяется фильтром безопасности. На основании роли принимается решение о допуске к тем или иным маршрутам. Такой подход позволяет надёжно разделить публичные сценарии просмотра каталога и закрытые сценарии управления площадками и заявками.

Для доступа к данным используются репозитории Spring Data JPA. Это упрощает реализацию типовых операций и уменьшает объём инфраструктурного кода. Сущности предметной области аннотируются для отображения на таблицы PostgreSQL, а связи между ними определяются через внешние ключи. Валидация входных данных выполняется как на уровне DTO, так и в сервисах. Например, проверяется корректность адреса электронной почты, непустота обязательных полей, допустимость временного диапазона и отсутствие конфликтов с уже подтверждёнными бронированиями.

3.3 Реализация клиентской части платформы

Клиентская часть веб-приложения разработана как одностраничное приложение на React. Основой пользовательского интерфейса является набор функциональных компонентов, объединённых маршрутизацией React Router. Страницы каталога и карточек площадок доступны всем пользователям, тогда как сценарии управления объектами и заявками открываются после авторизации. Такой подход обеспечивает удобное разделение публичного и защищённого интерфейса без дублирования логики отображения.

Компонент каталога получает с сервера список лофтов и отображает его в виде карточек с краткой информацией. При переходе в карточку площадки пользователь видит развернутые сведения об объекте, фотографии и форму выбора интервала аренды. Изменение даты или времени инициирует пересчёт стоимости. В реализации возможно два варианта: локальный предварительный расчёт на основании тех же правил, что и на сервере, либо запрос к выделенному API-калькулятору. В любом случае интерфейс должен оперативно отражать изменение цены и, желательно, показывать применённые надбавки отдельно, повышая тем самым прозрачность расчёта.

Форма отправки заявки содержит поля имени и телефона. Перед отправкой выполняется клиентская валидация, предотвращающая неполное заполнение формы. После успешного ответа сервера пользователь получает подтверждение, а интерфейс очищает либо обновляет состояние формы. В данном сценарии важна обработка ошибок: если интервал уже недоступен или введены некорректные данные, пользователь должен получить понятное сообщение без технических деталей реализации.

```

11 export default function RegisterLandlordModal({ onClose, onSuccess }: Props) : Element { Show usages
12   const [form, setForm] = useState({
13     email: '', password: '', name: '', metroStation: '', city: '', phone: '',
14     address: '', minutesFromMetro: '', maxSeats: '', area: '', baseHourlyRate: '',
15     description: '', loftEmail: '', photoUrls: '',
16   })
17   const [error, setError] = useState( initialState: '' )
18   const [loading, setLoading] = useState( initialState: false )
19
20   const set : (key: string) => (e: ChangeEvent<HTMLInput... = (key: string) : (e: ChangeEvent<HTMLInputElement | HTMLTe... => (e: React.C
21     setForm(f : { email: string; password: string; name: ... => ({ ...f, [key]: e.target.value })))
22
23   const submit : (e: FormEvent<Element>) => Promise<void> = async (e: React.FormEvent) : Promise<void> => { Show usages
24     e.preventDefault()
25     setLoading( value: true ); setError( value: '' )
26     try {
27       const photoUrls : string[] = form.photoUrls
28       ? form.photoUrls.split( separator: '\n' ).map(s : string => s.trim()).filter(Boolean)
29       : []
30       const data : any = await registerLandlord({
31         email: form.email, password: form.password,
32         name: form.name, metroStation: form.metroStation,
33         city: form.city, phone: form.phone, address: form.address,
34         minutesFromMetro: Number(form.minutesFromMetro),
35         maxSeats: Number(form.maxSeats),
36         area: Number(form.area),
37         baseHourlyRate: Number(form.baseHourlyRate),
38         description: form.description || undefined,
39         loftEmail: form.loftEmail || undefined,
40         photoUrls: photoUrls.length ? photoUrls : undefined,
41       })
42       localStorage.setItem( key: 'token', data.token )
43       onSuccess({ email: data.email, role: data.role, token: data.token })
44     } catch (err: unknown) {
45       const e = err as { response?: { data?: { error?: string } } }
46       setError(e.response?.data?.error || 'Ошибка регистрации')
47     } finally { setLoading( value: false ) }
48   }
49
50   const inputCls = "w-full px-3 py-2.5 border border-slate-300 rounded-lg text-sm focus:outline-none focus:ring-2
51
52   return (
53     <Modal title="Добавить площадку" onClose={onClose}>
54       <form onSubmit={submit} className="space-y-4">
55         {error && <p className="text-sm text-red-500 bg-red-50 px-3 py-2 rounded-lg">{error}</p>}
56         <div className="space-y-1.5">
57           <label className="text-sm font-medium text-slate-700">Email</label>
58           <input type="email" value={form.email} onChange={set( key: 'email')} required placeholder="email@example.com" />
59         </div>

```

Рисунок 3.2 – Фрагмент кода формы добавления новой площадки

Интерфейс регистрации арендатора и арендодателя реализован отдельно в соответствии с логикой бизнес-процессов. Для арендатора предусмотрена классическая регистрация по email и паролю. Для арендодателя точкой входа является сценарий добавления площадки, при котором создаётся и аккаунт владельца, и первый объект размещения. После входа арендодатель видит собственные площадки, кнопку добавления нового объекта и раздел заявок.

Список заявок отображается с бейджем новых обращений, что повышает информативность интерфейса и сокращает время реакции владельца.

Технологически клиентская часть использует Axios для централизованной настройки HTTP-взаимодействия. В запросах к защищённым маршрутам автоматически добавляется JWT-токен. TypeScript-типизация объектов запросов и ответов снижает вероятность ошибок интеграции между frontend и backend. Использование TailwindCSS, React и набора утилит управления классами позволяет построить единообразный, аккуратный и достаточно лёгкий интерфейс, адаптированный к задачам прикладной торговой платформы.

ЗАКЛЮЧЕНИЕ

СПИСОК ЛИТЕРАТУРЫ

1. А.Р. Иваненко, Е.Г. Калабина Внедрение системы динамического ценообразования в ритейле: аналитический обзор практик российских компаний commerce [Электронный ресурс] // CyberLeninka. — URL: <https://cyberleninka.ru/article/n/vnedrenie-sistemy-dinamicheskogo-tsenoobrazovaniya-v-riteyle-analiticheskiy-obzor-praktik-rossiyskih-kompaniy> (дата обращения: 09.03.2026).
2. Котлер Ф., Келлер К.Л. Маркетинг-менеджмент. — 15-е изд. — СПб.: Питер, 2022. — 848 с.
3. Федотова Е.Л., Поспелова М.В. Информационные технологии и системы: учебное пособие. — М.: ФОРУМ, 2020. — 352 с.
4. Тарасевич В.М. Ценовая политика предприятия: учебник. — СПб.: Питер, 2021. — 304 с.
5. Ценообразование на основе данных: подходы data mining к динамическому ценообразованию в e-commerce [Электронный ресурс] // CyberLeninka. — URL: <https://cyberleninka.ru/article/n/tsenoobrazovanie-na-osnove-dannyh-podhody-data-mining-k-dinamicheskomu-tsenoobrazovaniyu-v-e-commerce> (дата обращения: 11.03.2026).
6. Совершенствование стратегий ценообразования для малого и среднего бизнеса в сфере электронной коммерции B2C [Электронный ресурс] // CyberLeninka. — URL: <https://cyberleninka.ru/article/n/sovershenstvovanie-strategiy-tsenoobrazovaniya-dlya-malogo-i-srednego-biznesa-v-sfere-elektronnoy-kommertsii-b2c-na-osnove> (дата обращения: 01.04.2026).
7. Разработка и применение модели динамического ценообразования для сглаживания конкуренции на маркетплейсах [Электронный ресурс] // CyberLeninka. — URL: <https://cyberleninka.ru/article/n/razrabotka-i>

- primenenie-modeli-dinamicheskogo-tsenoobrazovaniya-dlya-sglazhivaniya-konkurentsii-na-marketpleysah (дата обращения: 03.04.2026).
8. Динамическое ценообразование как инструмент управления спросом и прибылью [Электронный ресурс] // CyberLeninka. — URL: <https://cyberleninka.ru/article/n/dinamicheskoe-tsenoobrazovanie-kak-instrument-upravleniya-sprosom-i-pribylyu> (дата обращения: 13.03.2026).
 9. Java SE 17 Documentation [Электронный ресурс] // Oracle. — URL: <https://docs.oracle.com/en/java/javase/17/> (дата обращения: 09.03.2026).
 10. PostgreSQL 16 Documentation [Электронный ресурс] // PostgreSQL. — URL: <https://www.postgresql.org/docs/16/> (дата обращения: 02.03.2026).
 11. React Documentation [Электронный ресурс] // React. — URL: <https://react.dev/> (дата обращения: 01.03.2026).
 12. TypeScript Documentation [Электронный ресурс] // TypeScript. — URL: <https://www.typescriptlang.org/docs/> (дата обращения: 18.03.2026).
 13. Tailwind CSS Documentation [Электронный ресурс] // Tailwind CSS. — URL: <https://tailwindcss.com/docs> (дата обращения: 03.04.2026).
 14. React Router Documentation [Электронный ресурс] // React Router. — URL: <https://reactrouter.com/home> (дата обращения: 11.03.2026).
 15. CVA (Class Variance Authority) [Электронный ресурс] // CVA. — URL: <https://cva.style/> (дата обращения: 04.03.2026).

ПРИЛОЖЕНИЯ